ICG:
Aula de abertura

Profª. Tainá Isabela

Apresentação

Acadêmica:

- Técnico em Informática - IFMT
- Bacharel em Ciências da Computação - UFMT
- Pós em BI, Big Data e Inteligência Artificial

Profissional:

- Desenvolvedora Back-end desde 2021;

Plano de Ensino

Ementa

- Transformações Geométricas em duas e três dimensões;
- Coordenadas Homogêneas e Matrizes de Transformação;
- Transformação entre Sistemas de Coordenadas 2D e 3D;
- Recorte;
- Transformações de Projeção: Paralela e Perspectiva;
- Câmera Virtual;
- Definição de Objetos e Cenas Tridimensionais: Modelos Poliedrais e Malhas de Polígonos;
- O Processo de Rendring: Fontes de Luz;
- Remoção de Linhas e Superfícies Ocultas;
- Modelos de Iluminação e de Tonalização (shading);
- Raytracing;
- Aplicação de Texturas;

Avaliação

Questionários e Exercícios (Peso 3,0)

Atividades ao longo do curso para fixação do conteúdo.

Trabalho Prático (Peso 3,0)

Trabalhos (T1 e T2) envolvendo atividades aplicadas, estudos dirigidos, projetos ou produções correlatas aos conteúdos ministrados;

Prova Final (Peso 4,5)

Provas (P1 e P2) destinadas à verificação do domínio dos conteúdos teóricos e práticos abordados na disciplina;

Bibliografia

- CONCI, Aura; AZEVEDO, Eduardo; LETA, Fabiana R. Computação gráfica. Rio de Janeiro: Elsevier: Campus, c2003-2008. 2 v. ISBN 9788535212525 (v. 1)

- HEARN, D.; BAKER, M.P. Computer Graphics with OpenGL, Prentice Hall; 3ª edição, 2003.

- A. K. Peters, Fundamentals of Computer Graphics, Fourth Edition, 2016, ISBN:1482229390 9781482229394.

ICG:
Introdução a Computação Gráfica

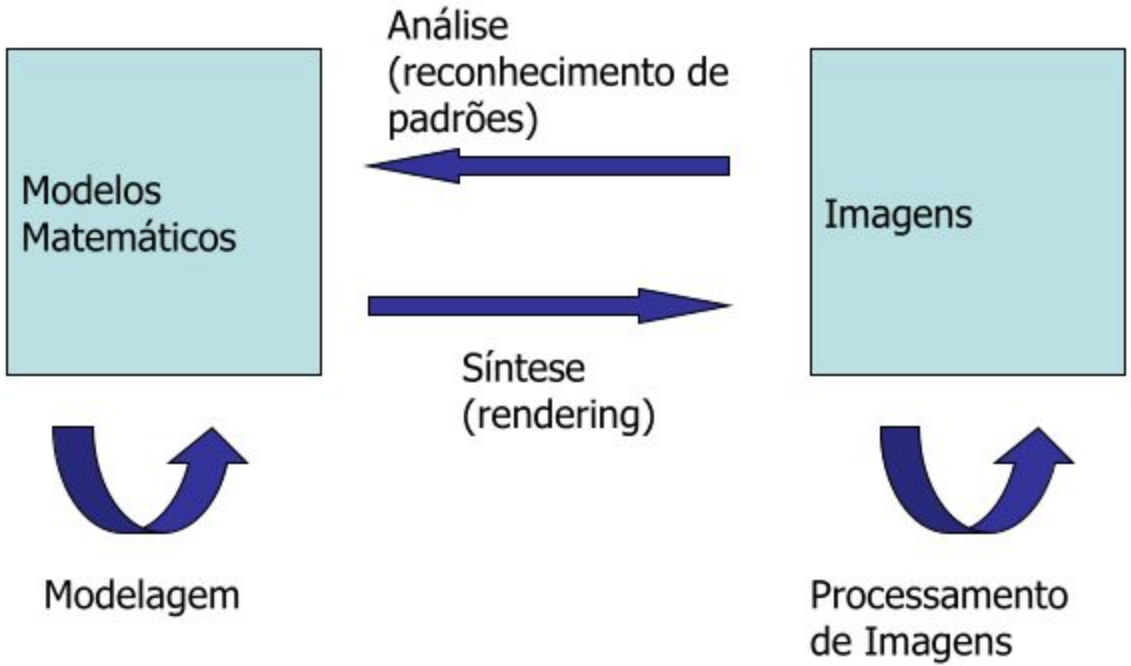
Profª. Tainá Isabela

Computação Gráfica

Computação Gráfica é a disciplina que trata das técnicas e dos métodos computacionais, que convertem dados para dispositivos gráficos e vice-versa.

Também é possível definir em:

Computação Gráfica é o veículo de comunicação homem/máquina mais adequado à percepção humana.


Análise
(reconhecimento de
padrões)
Modelos
Matemáticos
Imagens
Síntese
(rendering)
Modelagem
Processamento
de Imagens

Computação Gráfica

Como **visualização** entende-se que a imagem é gerada através de um modelo matemático que contém os elementos gráficos básicos.

Também conhecida como **Computação Gráfica Generativa**.

- Engenharia;
- Geologia;
- Cartografia;

Computação Gráfica

O **Processamento de Imagens** busca o realismo da imagem digital, tentando torná-la mais acessível à percepção humana.

- Pesquisa Biológicas;
- Interpretações de satélites;
- Medicina;
- Filmes;

Computação Gráfica

A **visão** baseia-se na obtenção da descrição da imagem digital, fornecida comumente por um conversor analógico/digital **(Reconhecimento de Padrões**).

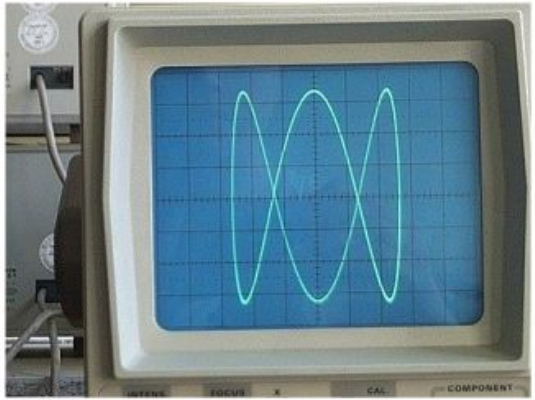
Por exemplo tem-se a “visão” de um robô industrial e a “leitura” por computador de caracteres manuscristos.

Quando se processa somente os dados e não há geração de imagens tem-se uma outra área dentro da informática que então é denominada de **Processamento de Dados**.

1950

Início da Computação Gráfica, pela primeira vez, um tubo de raios catódicos é ligado à um Computador, on MIT e Projeto Sage, onde um radar foi ligado à um Computador, num projeto do exército americano, utilizando interação com Light Pen;

- Plotter e Osciloscópios;



1960

No MIT o projeto TX2 com o aluno Ivan Sutherland usa um computador para fazer desenhos em sua tese de doutorado;

- Monitores Vetoriais;
- Sketchpad

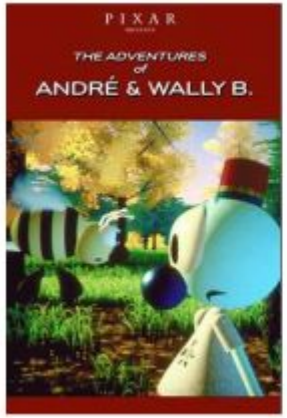
1970

Surgem os primeiros monitores à varredura.

- Monitor CRT (Cathode Ray Tube);
- Star Wars;

1980

- CRT colorido
- Silicon Graphics, Inc. (SGI)
- Pixar
- Star Trek 2
- TRON



1990

- Hardware gráfico
- OpenGL
- Toy Story
- Unreal Engine
- Star Wars Episode I

2000

- Shaders
- Tessellation
- Direct3D 9.0

2010

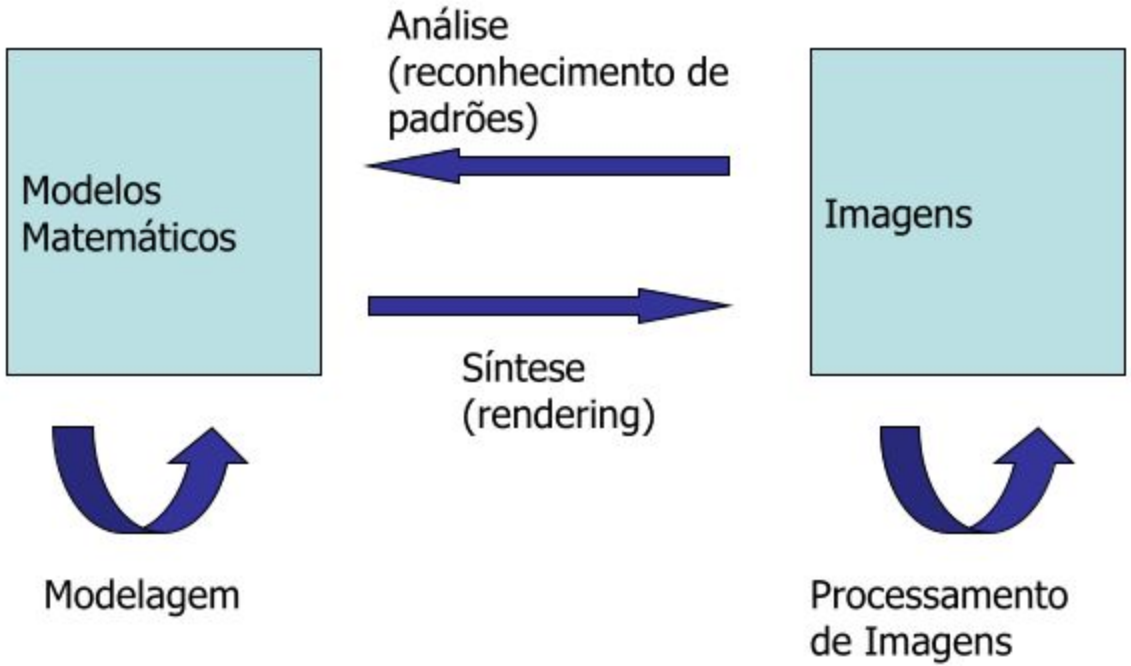
- Ray Tracing em tempo real
- Vulkan (AMD, Khronos)

2020

- VIrtual Sets
- CG Mobile
- Realidade Virtual
- Hardware Avançado
- Geração de CG com uso de IA

Conteúdo da disciplina

1. OpenGL 101
2. Modelagem e Blender
3. Transformações
4. Geometria Projetiva e Câmera
5. Luz e Cores
6. Iluminação clássica
7. Shaders e GLSL
8. Texturas
9. Animação
10. Colisão
11. Partículas e Ray Tracing


Análise
(reconhecimento de
padrões)
Modelos
Matemáticos
Imagens
Síntese
(rendering)
Modelagem
Processamento
de Imagens

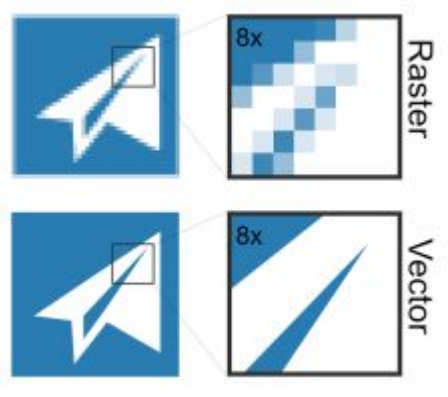
Representações Vetoriais

Permitem uma série de operações sem (quase) perda de precisão.
Por que “quase”? Estruturas de dados utilizam pontos e vetores cujas coordenadas são números reais, portanto **é necessário usar aproximações**

- Representação em ponto-flutuante
- Números racionais

Complexidade de processamento = O (no vértices / vetores)
Exibição

- Dispositivos vetoriais
- Dispositivos matriciais (requer amostragem, i.e., rasterização)



Representações Matriciais

Representação flexível e muito comum

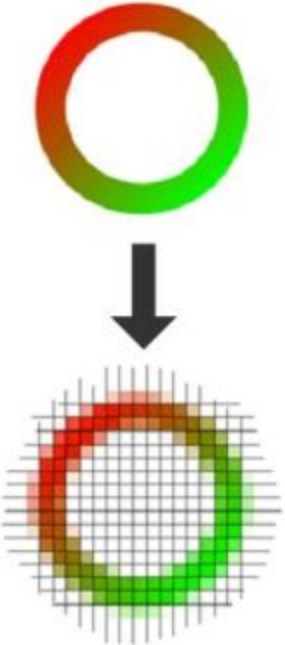
Complexidade de processamento = O (n de pixels)

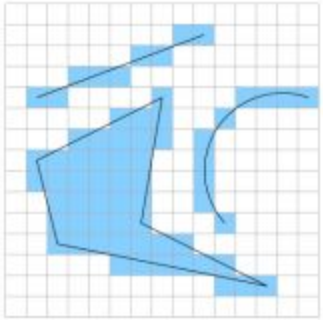
Muitas operações implicam em perda de precisão (reamostragem)

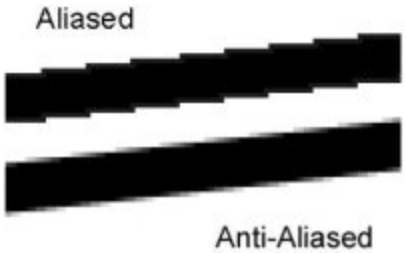
- Ex.: rotação, escala
- Técnicas para lidar com o problema
    - Ex.: técnicas anti-serrilhado (anti-aliasing)

Exibição

- Dispositivos matriciais
- Dispositivos vetoriais (requer uso de técnicas de reconhecimento de padrões)






Aliased
Anti-Aliased

OpenGL

O OpenGL é uma API para geração de gráficos

- 3D e 2D
- Primitivas vetoriais e matriciais (imagens)
- Capaz de gerar imagens de alta qualidade
- Independente de plataforma
- Independente de sistema de janelas

Sistemas de Janela

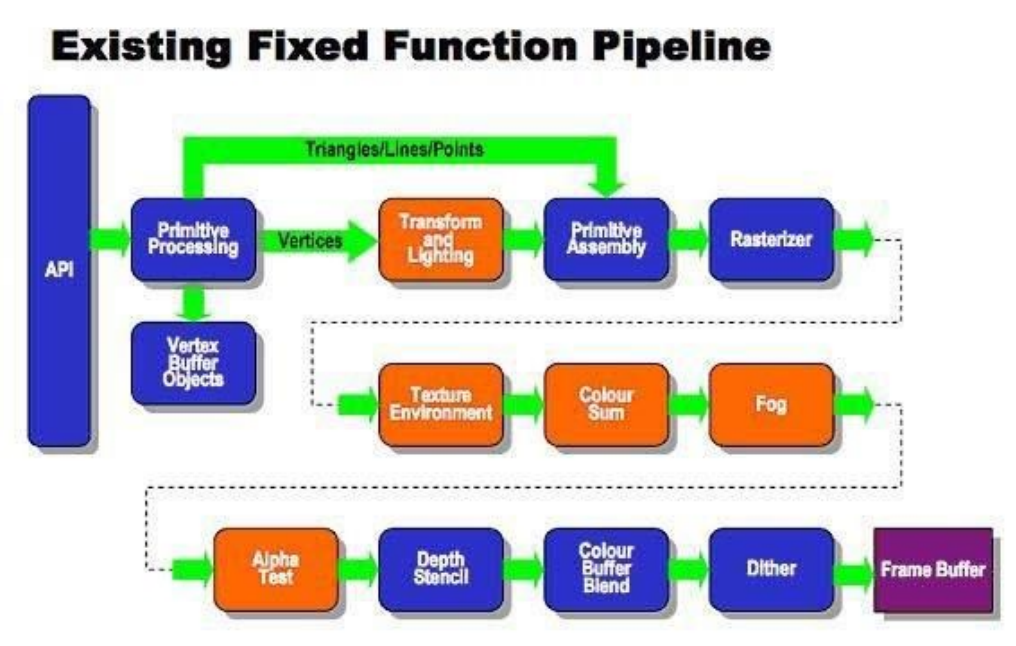
- Principal meio de interação homem/máquina em ambientes de computação modernos
- Tela é dividida em janelas (eventualmente superpostas)
- Janelas são controladas por aplicações que têm a incumbência de mantê-las sempre atualizadas
- Interação do usuário e do próprio sistema de janelas são comunicados à aplicação através de eventos. Ex.:
    - Mouse foi apertado
    - Janela foi redimensionada

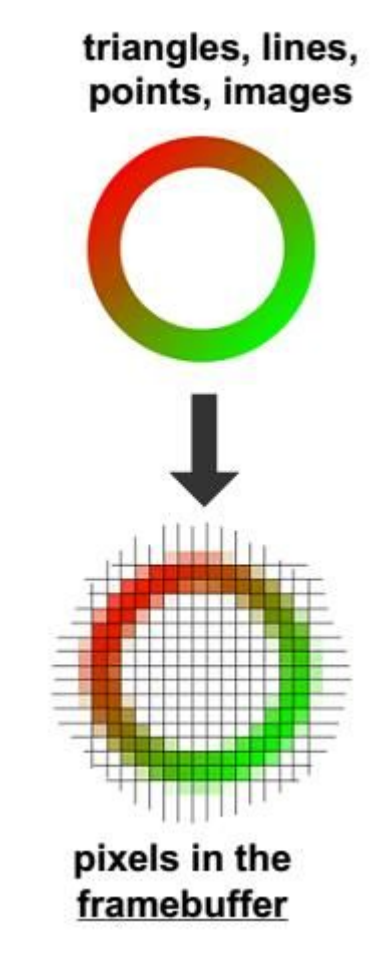
Sistemas de Janela

- Eventos são tratados por rotinas callback da aplicação. Ex.:
  - Redesenhar o conteúdo da janela
  - Mover um objeto de um lado para outro da janela

- Cada Sistema de Janelas possui uma API distinta:
  - MS Windows, X Apple
  - Portabilidade: Camada de interface com diversos SJ mas com API única (ex.: GLUT)

Arquitetura do OpenGL





Desenhando com OpenGL

OpenGL funciona como uma máquina de estados

- API tem rotinas para
  - Desenhar primitivas geométricas e imagens
  - Alterar variáveis de estado (ex.: cor, material, fontes de iluminação, etc)
  - Consultar variáveis de estado

- OpenGL é um padrão em evolução
  - Mecanismo padronizado de extensões
  - Novas versões são estabelecidas por um comitê (ARB) de usuários e fabricantes

APIs Relacionadas

- GLU (OpenGL Utility Library)
  - Parte do padrão OpenGL
  - NURBS, trianguladores, quádricas, etc.

- AGL, GLX, WGL
  - Camadas entre o OpenGL os diversos sistemas de janelas

- GLUT (OpenGL Utility Toolkit)
  - API portátil de acesso aos sistemas de janelas
  - Encapsula e esconde as camadas proprietárias
  - Não é parte oficial do OpenGL

Anatomia de um programa OpenGL/GLUT

```
#include <GL/glut.h>
/* Outros headers */

void display (void) {
    ...
}
/* Outras rotinas callback */

int main (int argc, char *argv[]) {
    glutInit (&argc, argv);
    glutInitDisplayMode( modo );
    glutCreateWindow( nome_da_janela );
    glutDisplayFunc( display );
    glutReshapeFunc( reshapeCallback );
    /* Registro de outras rotinas callback */
    glutMainLoop();
    return 0;
}
```

Headers

Rotinas Callback

Inicialização do GLUT

Inicialização da janela

Registro de callbacks

Laço principal

GLUT – Registrando Callbacks

- Callbacks são rotinas que serão chamadas para tratar eventos.
- Para uma rotina callback ser efetivamente chamada ela precisa ser registrada através da função
  - glutXxxFunc (callback)

Onde **Xxx** designa uma classe de eventos e callback é o nome da rotina

- Por exemplo, para registrar uma callback de desenho chamada ***Desenho***, usa-se:
  - glutDisplayFunc (Desenho);

GLUT – Callback de desenho

É a rotina chamada automaticamente sempre que a janela ou parte dela precisa ser redesenhada (ex.: janela estava obscurecida por outra que foi fechada)

- Todo programa GLUT precisa ter uma!
- Exemplo:

```
void display ( void )
{
  glClear( GL_COLOR_BUFFER_BIT );
  glBegin( GL_TRIANGLE_STRIP );
    glVertex3fv( v[0] );
    glVertex3fv( v[1] );
    glVertex3fv( v[2] );
    glVertex3fv( v[3] );
  glEnd();
  glutSwapBuffers(); /* Usamos double-buffering! */
}
```

GLUT – Callback de redimensionamento

Chamada sempre que a janela é redimensionada, isto é, teve seu tamanho alterado.

- ***void reshape (int width, int height){...}***
- width/height são a nova largura/altura da janela (em pixels)

Se uma rotina de redimensionamento não for especificada, o GLUT usa uma rotina de redimensionamento “default” que simplesmente ajusta o viewport para usar toda a área da janela

GLUT - Callbacks

Outras callbacks comunmente usadas

void keyboard (unsigned char key, int x, int y)

- Eventos de teclado

void mouse(int button, int state, int x, int y)
void motion(int x, int y)
void passiveMotion(int x, int y)

- Eventos de mouse

void idle (void)

- Chamada continuamente quando nenhum outro evento ocorre

Programa OpenGL/GLUT - Inicialização

- Inicialização do GLUT

glutInit (int* argc, char** argv)

  - Estabelece contato com sistema de janelas
  - Em X, opções de linha de comando são processadas e removidas

Programa OpenGL/GLUT - Inicialização

Inicialização da(s) janela(s)

◦ glutInitDisplayMode (int modo)

• Estabelece o tipo de recursos necessários para as janelas que serão criadas. Modo é um "ou" bit-a-bit de constantes:

– **GLUT_RGB** cores dos pixels serão expressos em RGB
– **GLUT_DOUBLE** bufferização dupla (ao invés de simples)
– **GLUT_DEPTH** buffer de profundidade (z-buffer)
– **GLUT_ACCUM** buffer de acumulação
– **GLUT_ALPHA** buffer de cores terá componente alfa

Programa OpenGL/GLUT - Inicialização

- glutInitWindowPosition (int x, int y)
  - Estabelece a posição inicial do canto superior esquerdo da janela a ser criada
- glutInitWindowSize (int width, height)
  - Estabelece o tamanho (em pixels) da janela a ser criada

Programa OpenGL/GLUT - Inicialização

Criação da(s) janela(s)

◦ int glutCreateWindow (char* nome)

- Cria uma nova janela primária (top-level)
- Nome é tipicamente usado para rotular a janela
- O número inteiro retornado é usado pelo GLUT para identificar a janela

•Outras inicializações

- Após a criação da janela é costumeiro configurar variáveis de estado do
- OpenGL que não mudarão no decorrer do programa. Por exemplo:
  - Cor do fundo
  - Tipo de sombreamento de desejado

Programa OpenGL/GLUT – Laço Principal

Depois de registradas as callbacks, o controle é entregue ao sistema de janelas:

```
glutMainDisplayLoop (void)
```

- Esta rotina na verdade é o “despachante” de eventos
- Ela nunca retorna

OpenGL – Primitivas de desenho

glBegin (PRIMITIVA)

- especificação de vértices, cores, coordenadas de textura, propriedades de material

glEnd ()

- Entre **glBegin()** e **glEnd()** apenas alguns comandos podem ser usados. Ex.:

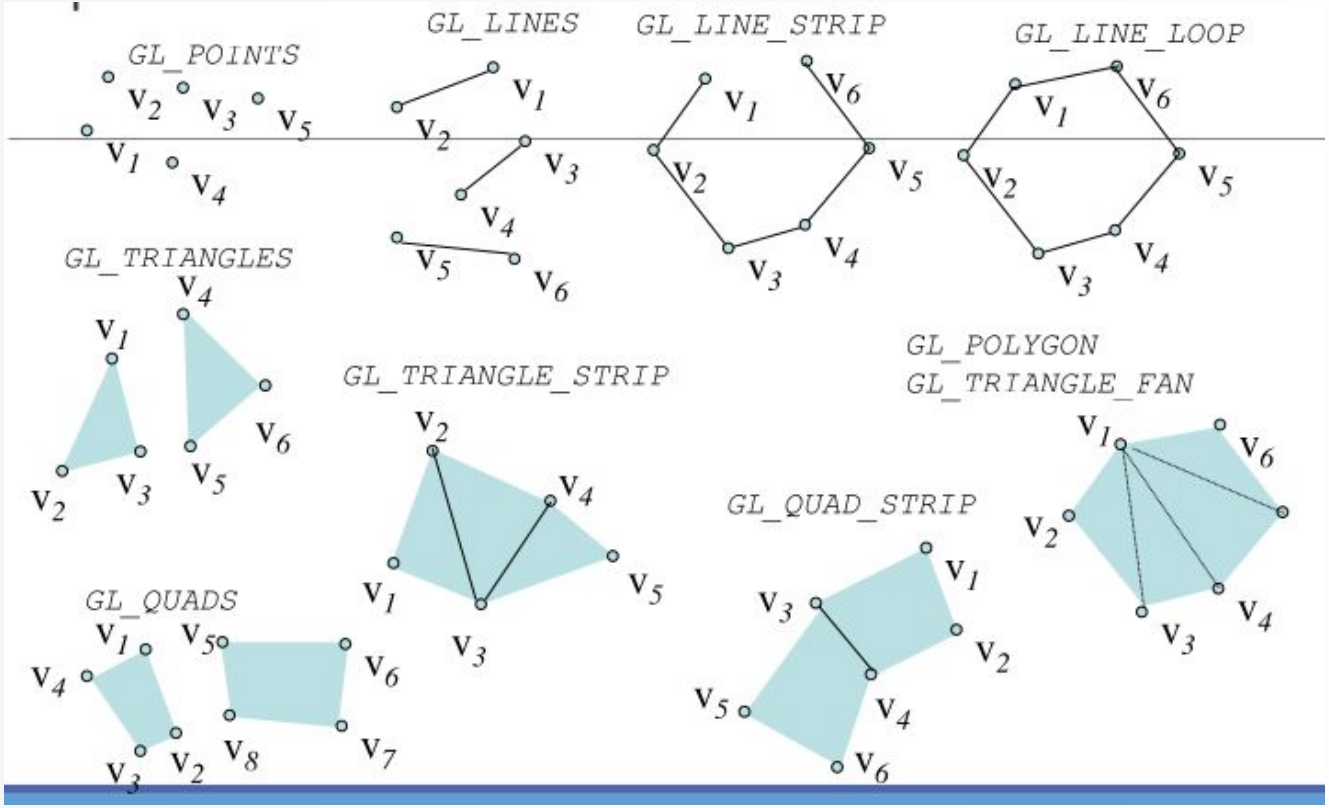
  glMaterial
  glNormal
  glTexCoord

OpenGL – Primitivas de desenho

Uma vez emitido um vértice (**glVertex**), este é desenhado com as propriedades (cor, material, normal, coordenadas de textura etc) registradas nas variáveis de estado correspondentes

Em resumo: Antes de emitir um vértice, assegurar-se que cor, material, normal, etc têm o valor certo

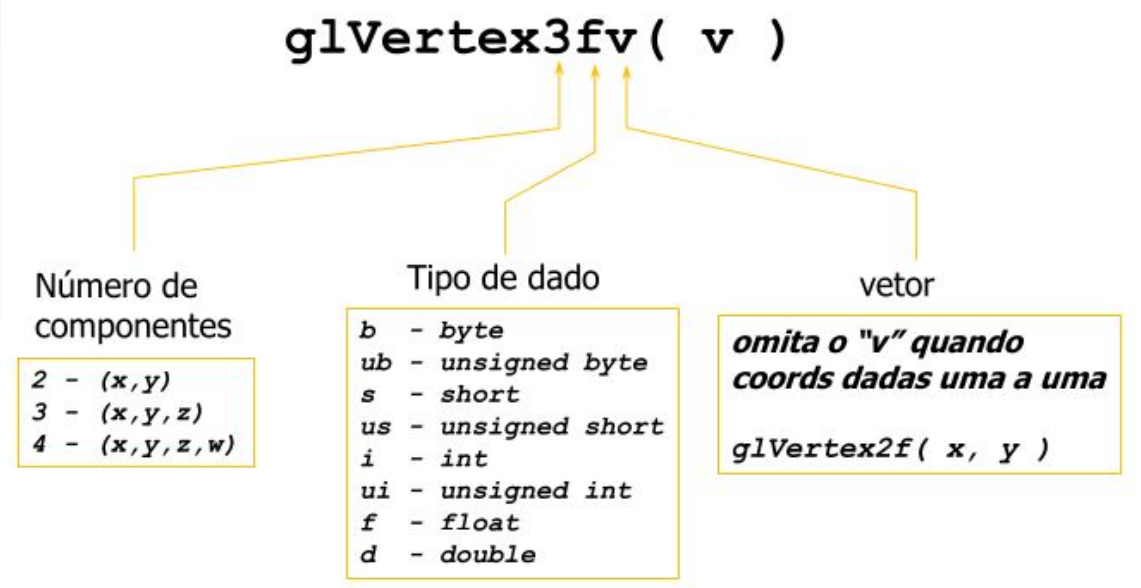
OpenGL – Primitivas de desenho



OpenGL – Exemplo de desenho simples

```
void drawRhombus( GLfloat color[] )
{
glBegin( GL_QUADS );
glColor3fv( color );
glVertex2f( 0.0, 0.0 );
glVertex2f( 1.0, 0.0 );
glVertex2f( 1.5, 1.118 );
glVertex2f( 0.5, 1.118 );
glEnd();
}
```

OpenGL – Convenções de Nome



Questionário

Prática

Instale Python3 e todas as bibliotecas necessárias para rodar o OpenGL, usando o código ofertado para testes.

sudo apt install python3-pip

pip install pywavefront pyglet